Project 2 – Acme Analytics Linux Platform

Day 4 · Phase 1A wrap-up and Phase 2: LVM, NFS, backups and restore

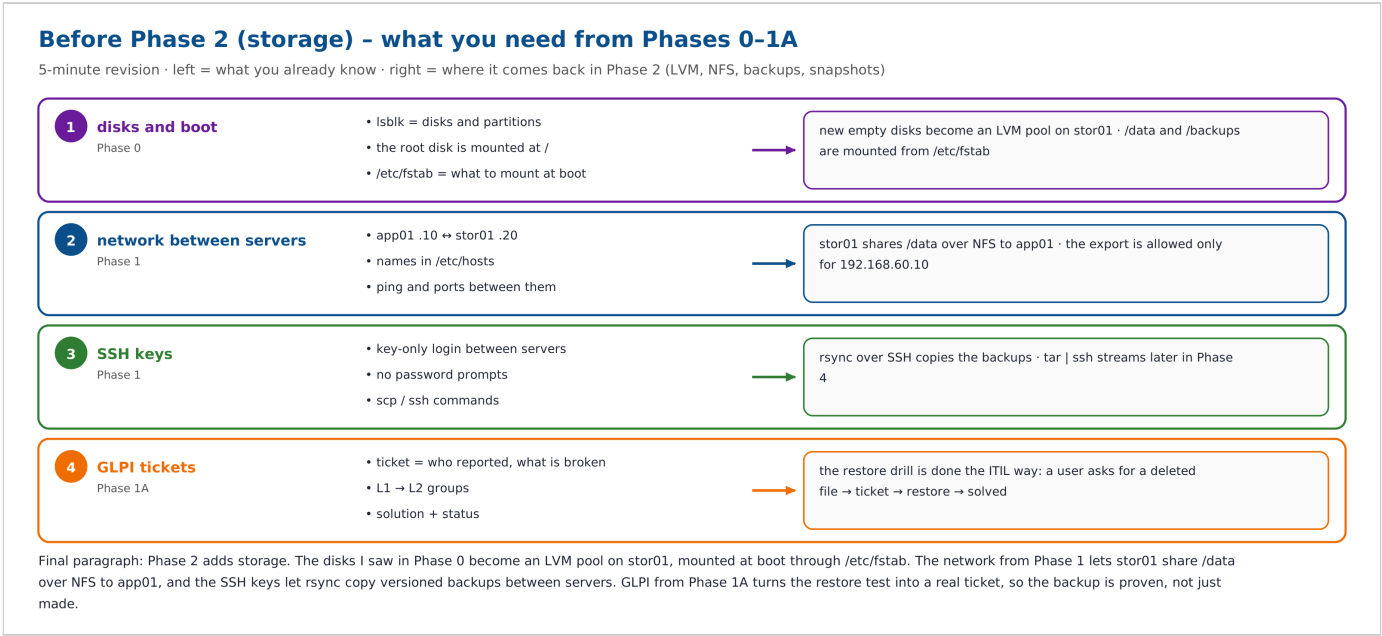
Muhammad Bilal Javed · 18 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	1A – Service desk: GLPI tickets, knowledge base and asset inventory
Hosts worked on	svc01 (GLPI), stor01 (LVM, NFS server), app01 (NFS client)
Status	Phase 1A ✅ for now – default accounts disabled, demo data off, Change #1 recorded. Entity, SLAs and users (Step 1A.3) added later, when the first tickets need them · Phase 2 🟡 – Steps 2.1 (LVM), 2.2 (NFS) and 2.3 (backups + restore through a ticket) ✅; LVM snapshot next
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary.

Before Phase 2 – quick revision (Phases 0–1A)

💡 **5 minutes:** what I already know from Phases 0–1A and where it comes back in Phase 2.



Before Phase 2 – quick revision

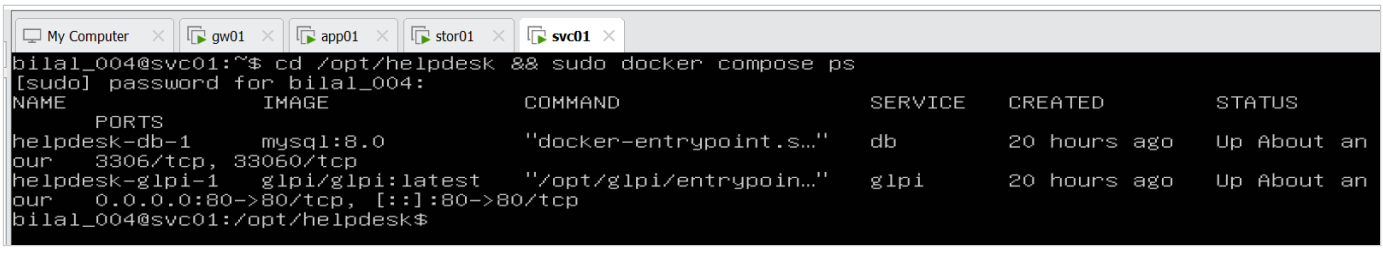
---

1. Start of the day – does GLPI survive a full power-off?

💡 **In plain words:** last night all four VMs were shut down. Today they were started again, gw01 first (it is the router). GLPI came back by itself – nobody had to start it.

```
cd /opt/helpdesk && sudo docker compose ps     # both containers "Up"
```

📸 **Evidence – containers created 20 hours ago, "Up about an hour" (since svc01 booted):**



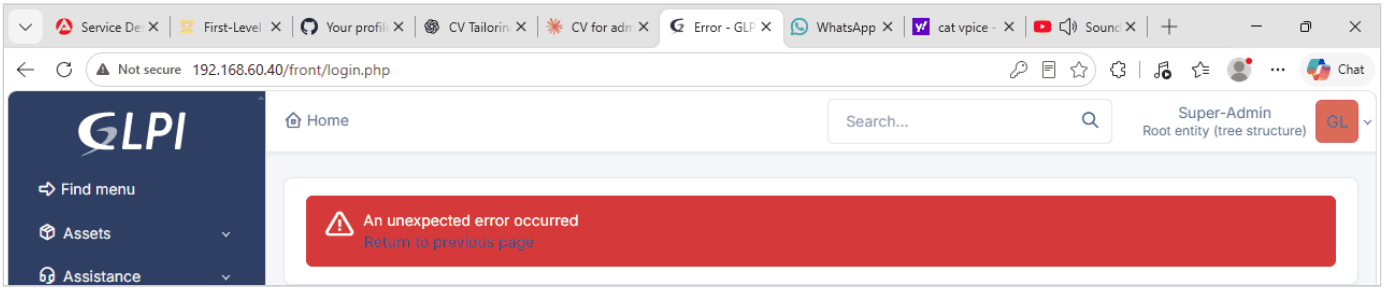
containers back after reboot

Why: `restart: unless-stopped` in `compose.yaml` tells Docker to start the containers again whenever the server boots. A service that needs a person to start it after every reboot is an outage waiting to happen.

Issue – “An unexpected error occurred”

Seen	The browser tab left open from last night, on `login.php`, showed *“An unexpected error occurred”* – while the top right already showed *Super-Admin*.
Cause	The login form carries a one-time security code (CSRF token). After the restart it was no longer valid. GLPI itself was fine.
Fix	Open a fresh address: `http://192.168.60.40/` (or click **Home**).
If it comes back	`sudo docker compose logs glpi --tail 30` and `/opt/helpdesk/storage/glpi/log/php-errors.log`

📸 **Evidence:**

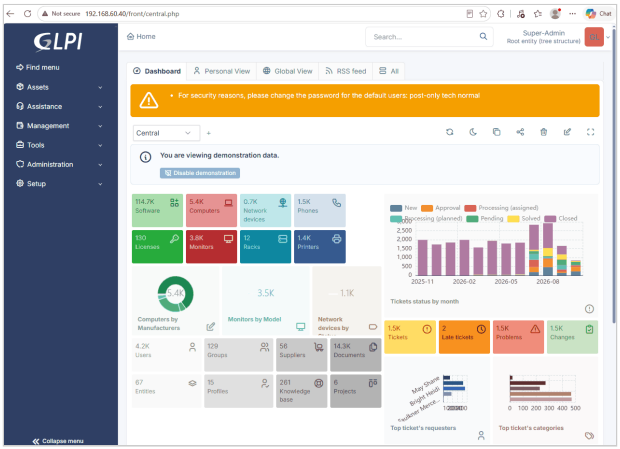


stale login tab

2. Clean-up – default accounts and demo data

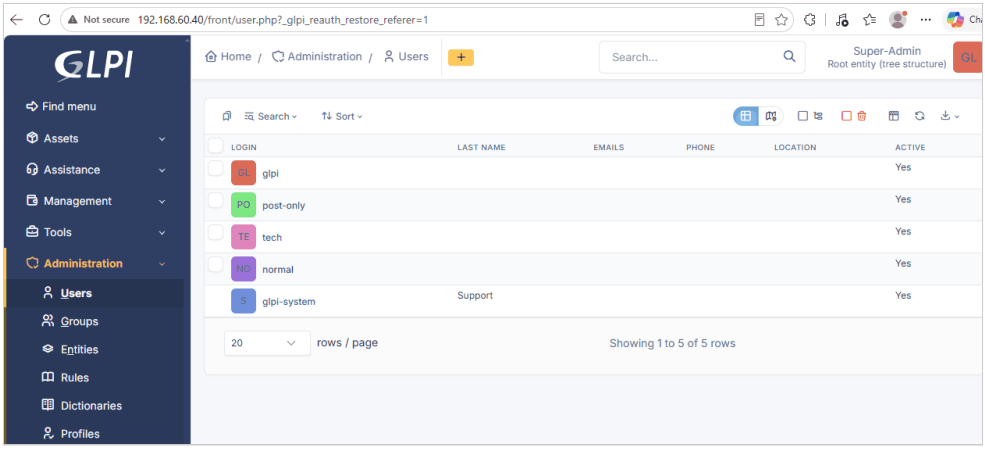
💡 **In plain words:** GLPI ships with four accounts whose passwords are printed in every guide on the internet. The `glpi` password was changed on Day 3; the other three are not needed, so they are switched off. The dashboard was also full of invented sample numbers.

📸 **Evidence – before: GLPI warns about the default passwords; “You are viewing demonstration data” (5.4K computers, 1.5K tickets – none of it real):**



dashboard before

📸 **Evidence – before: all five accounts Active:**

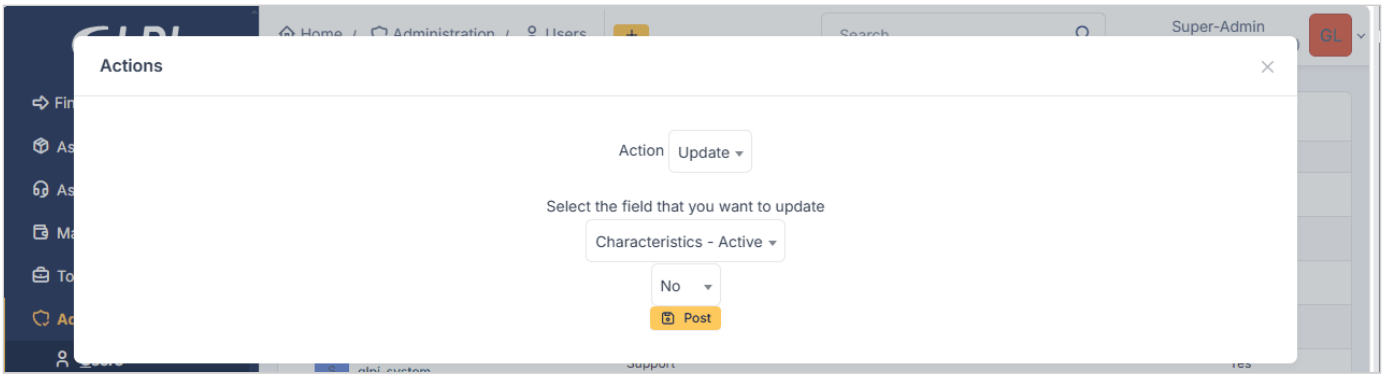


users before

Account	Action	Why
`glpi`	keep (password already changed)	my Super-Admin account
`post-only`, `tech`, `normal`	**Active: No**	default passwords known to everyone; not used
`glpi-system`	leave	GLPI's own account for automatic jobs; cannot be ticked

Administration → Users → tick the three → **Actions → Update → Characteristics – Active → No → Post**. Then Home → **Disable demonstration**.

📸 **Evidence – the mass update:**



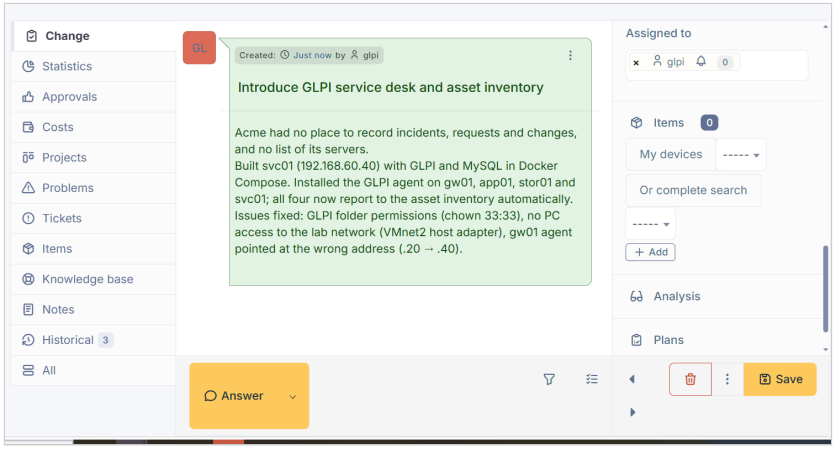
active no

3. The first ticket – Change #1

💡 **In plain words:** from now on, every piece of work is recorded in GLPI. The first record describes the service desk itself, written as a **Change**, because it was planned work with a way back.

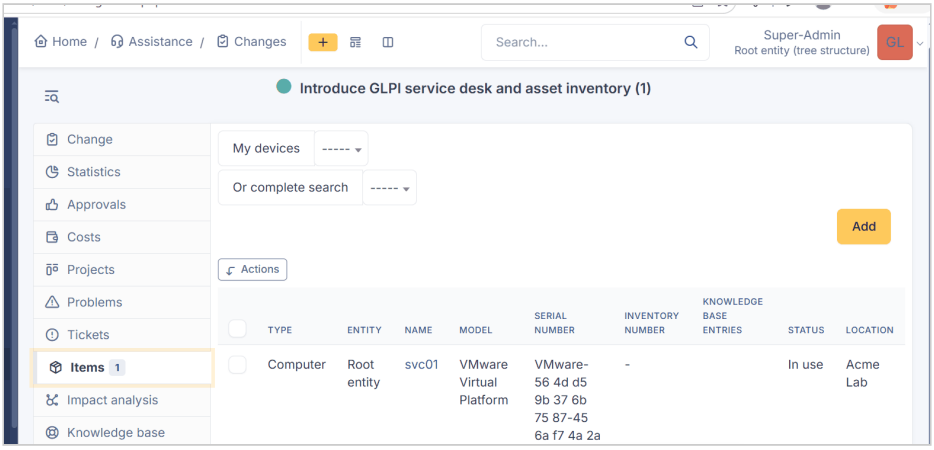
Field	Value
Title	Introduce GLPI service desk and asset inventory
Description	Acme had no place to record incidents, requests and changes, and no list of its servers. Built svc01 (192.168.60.40) with GLPI and MySQL in Docker Compose; GLPI agent on all four servers. Issues fixed: folder permissions (`chown 33:33`), no PC access to the lab network (VMnet2 host adapter), gw01 agent pointed at the wrong address (.20 → .40).
Item	svc01
Impacts	No downtime – a new server only; the existing servers only received a small agent
Control list	Containers Up; HTTP 200; 4 computers in Assets
Deployment plan	Docker + `compose up` → `chown 33:33` → VMnet2 host adapter → agent on 4 servers
Backout plan	`docker compose down`; restore snapshot `24-svc01-installed`; remove `glpi-agent` from the servers
Status	Closed

📸 **Evidence – Change #1 created:**



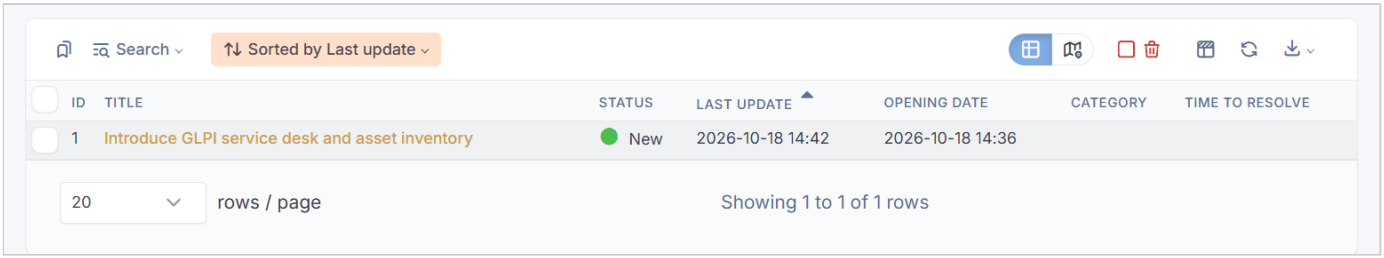
change created

📸 **Evidence – svc01 linked to the change (status In use, location Acme Lab – from the inventory):**



change item

📸 Evidence – Changes list, ID 1:



changes list

A change in GLPI can pass through New → Evaluation → Approval → Accepted → Test → Applied → Review → Closed. This one was already finished, so it goes straight to Closed.

4. ITIL basics – three ticket types and the SLA

Type	Use it for	Example in this project
Incident (Assistance → Tickets)	Something is broken	NFS mount hangs after a firewall change
Request (Assistance → Tickets, type Request)	Someone wants something	“Please give carol read access to /srv/shared”
Change (Assistance → Changes)	Planned work with a plan and a rollback	“Grow /data by 1 GB”

SLA (Service Level Agreement) – the promise of how fast IT reacts and fixes, by priority:

Priority	Example	Time to own	Time to resolve
High	App down, nobody can work	1 hour	8 hours
Medium	One user cannot reach a shared folder	4 hours	3 days
Low	“Please install a tool for me”	1 day	1 week

Time to own = how fast a technician must take the ticket (“I’m on it”). *Time to resolve* = how fast it must be fixed. GLPI starts both clocks when a ticket is created and marks it **late** when a promise is missed – the “2 Late tickets” on the demo dashboard were exactly that.

Key understanding (in my own words)

> In GLPI every piece of work is recorded as one of three types: an **Incident** when something is broken, a **Request** when someone asks for something, and a **Change** for planned work. A change carries extra boxes – impact, deployment plan and backout plan – so another engineer can see what was changed and how to undo it. My first change records the service desk itself, linked to svc01.
>
> An **SLA** (Service Level Agreement) is a written promise of how fast IT will react to a problem and fix it, depending on how serious it is. Every ticket gets a priority. For a high-priority problem, like the app being down, Acme promises that a technician takes the ticket within 1 hour (time to own) and fixes it within 8 hours (time to resolve); a medium problem gets 4 hours and 3 days. GLPI starts both clocks when the ticket is created and shows it as late when a promise is missed. So technicians know what to work on first and when to escalate, and managers can report how many tickets were solved on time. “Work according to SLAs” in a job ad means picking up tickets on time, handling the most urgent ones first, and escalating early before a deadline is missed.

5. Phase 2 – storage on stor01: LVM from empty disks

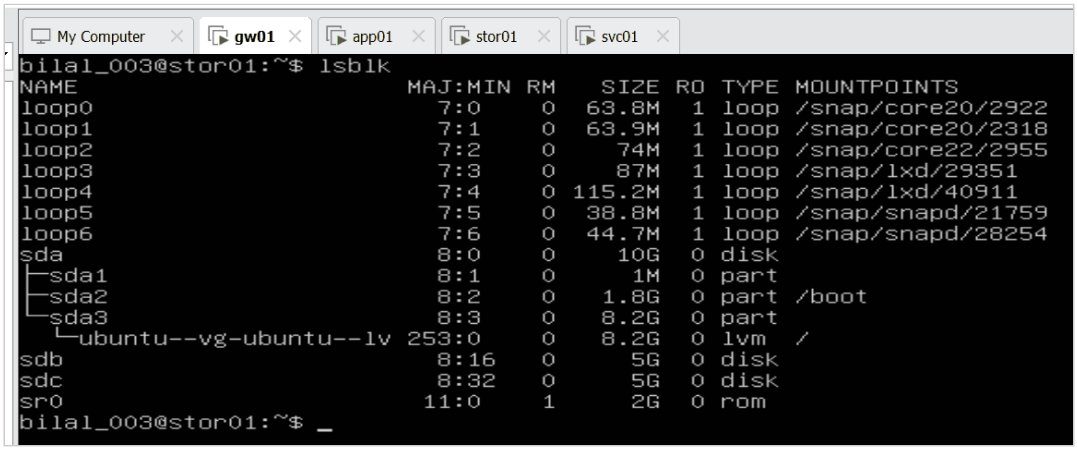
💡 **In plain words:** Acme’s application runs on **app01**, its data and backups live on **stor01**. stor01 needs storage that can **grow** when it fills up – without stopping anything. LVM gives exactly that: disks go into a pool, and volumes are cut from the pool and grown later.

```
sdb + sdc  →  pvcreate  →  PV, PV          (disks handed to LVM)
           →  vgcreate  →  VG "vgdata"     (one 10 GB pool)
           →  lvcreate  →  LV, LV          (slices: lvdata, lvbackups)
           →  mkfs.ext4 →  file system     (format)
           →  mount     →  /data, /backups (usable folders)
```

5.1 The disks

```
lsblk
```

📸 Evidence – sda = system disk (sda1/sda2 boot, sda3 = the system's own LVM); sdb and sdc = two empty 5 GB disks; loop0–6 are snap images, sr0 the CD drive:

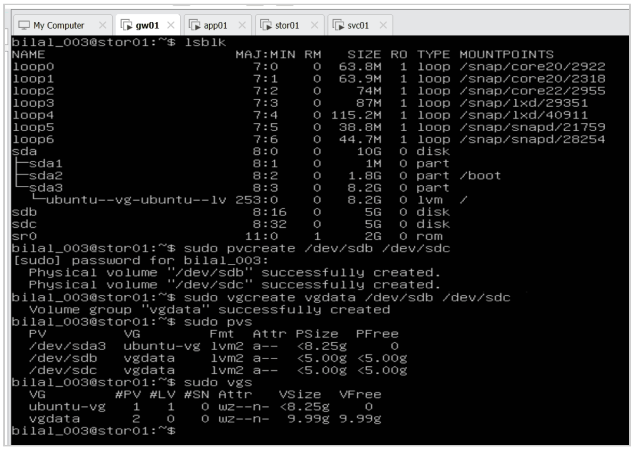


lsblk

5.2 Disks → physical volumes → volume group (the pool)

```
sudo pvcreate /dev/sdb /dev/sdc          # hand both disks to LVM (wipes them — check lsblk first)
sudo vgcreate vgdata /dev/sdb /dev/sdc   # one pool from both
sudo pvs                                 # the PVs
sudo vgs                                 # vgdata: 2 PVs, 0 LVs, 9.99 GB, all free
```

📸 Evidence:



pvcreate vgcreate

On svc01 the PV was a **partition** (sda3, because sda also holds the boot partitions); here the PVs are **whole disks** – they only store data. Both are normal.

5.3 Cut, format, mount

```
sudo lvcreate -L 3G -n lvdata vgdata       # 3 GB for app data
sudo lvcreate -L 3G -n lvbackup vgdata     # 3 GB for backups — about 4 GB stay free
sudo mkfs.ext4 /dev/vgdata/lvdata          # format
sudo mkfs.ext4 /dev/vgdata/lvbackup
```

📸 Evidence – 786,432 blocks × 4 KB = 3 GB; 196,608 inodes = the most files this volume can hold; the UUID is its permanent ID:

```
bilal_003@stor01:~$ sudo lvcreate -L 3G -n lvdata vgdata
  Logical volume "lvdata" created.
bilal_003@stor01:~$ sudo lvcreate -L 3G -n lvbackup vgdata
  Logical volume "lvbackup" created.
bilal_003@stor01:~$ sudo mkfs.ext4 /dev/vgdata/lvdata
mke2fs 1.46.5 (30-Dec-2021)
Creating filesystem with 786432 4k blocks and 196608 inodes
Filesystem UUID: a973da7a-5455-42c7-8608-58b604d1317c
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

bilal_003@stor01:~$ sudo mkfs.ext4 /dev/vgdata/lvbackup
mke2fs 1.46.5 (30-Dec-2021)
Creating filesystem with 786432 4k blocks and 196608 inodes
Filesystem UUID: cf760933-4488-4253-b6d3-59a6555d2732
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912

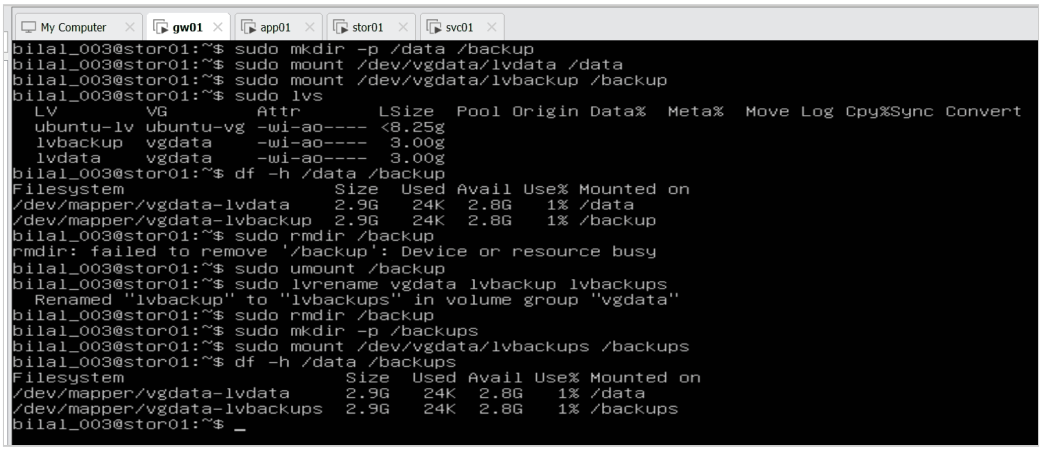
Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

lvcreate mkfs

The volume was first mounted on `/backup`; to keep the names consistent it became `/backups`, and the LV was renamed to match:

```
        sudo umount /backup                          # rmdir before this failed: "Device or resource busy"
        sudo lvrename vgdata lvbackup lvbackups      # renaming does not change the UUID
        sudo rmdir /backup && sudo mkdir -p /backups
        sudo mount /dev/vgdata/lvbackups /backups
        df -h /data /backups
# 2.9 GB each (ext4 keeps room for its journal, inode tables and 5 % for root)
```

📸 **Evidence:**



rename and mount

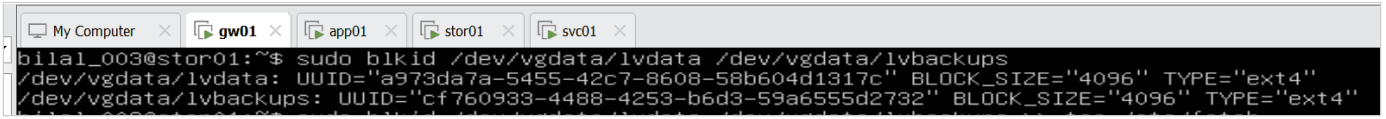
Lesson: a folder with a volume mounted on it is “busy” – `umount` first, then remove it. Separate volumes give isolation of space: full backups cannot fill the app’s `/data`.

5.4 Make the mounts permanent – `/etc/fstab`

`mount` lasts only until the next reboot. `/etc/fstab` is the list of what to mount at every boot.

```
        sudo blkid /dev/vgdata/lvdata /dev/vgdata/lvbackups      # the UUIDs
        sudo cp /etc/fstab /etc/fstab.bak                        # backup = way back
        echo 'UUID=a973da7a-5455-42c7-8608-58b604d1317c  /data     ext4  defaults  0 2' | sudo tee -a /etc/fstab
        echo 'UUID=cf760933-4488-4253-b6d3-59a6555d2732  /backups  ext4  defaults  0 2' | sudo tee -a /etc/fstab
```

📸 **Evidence – the UUIDs:**



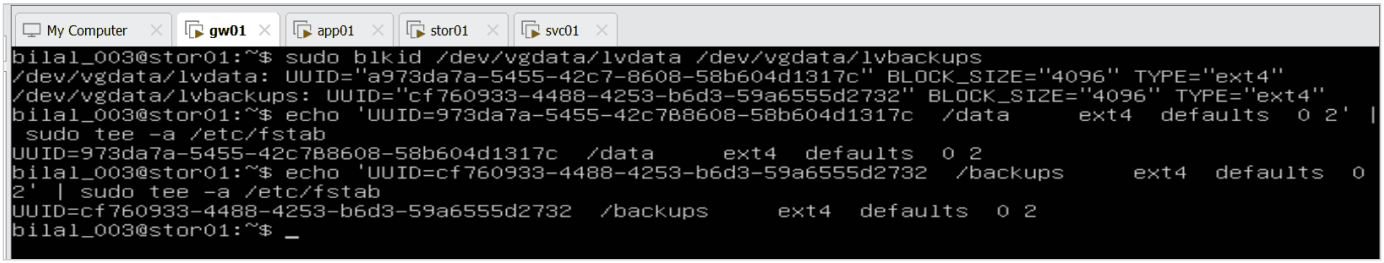
blkid

Column	Meaning
`UUID=…`	Which volume – never changes, even if disk names (sdb, sdc) do
`/data`	Where to mount it
`ext4`	File system type
`defaults`	Normal options (read/write …)
`0`	Old “dump” backup tool: off
`2`	Check the disk at boot, after / (which is 1)

Issue – one wrong character in a UUID

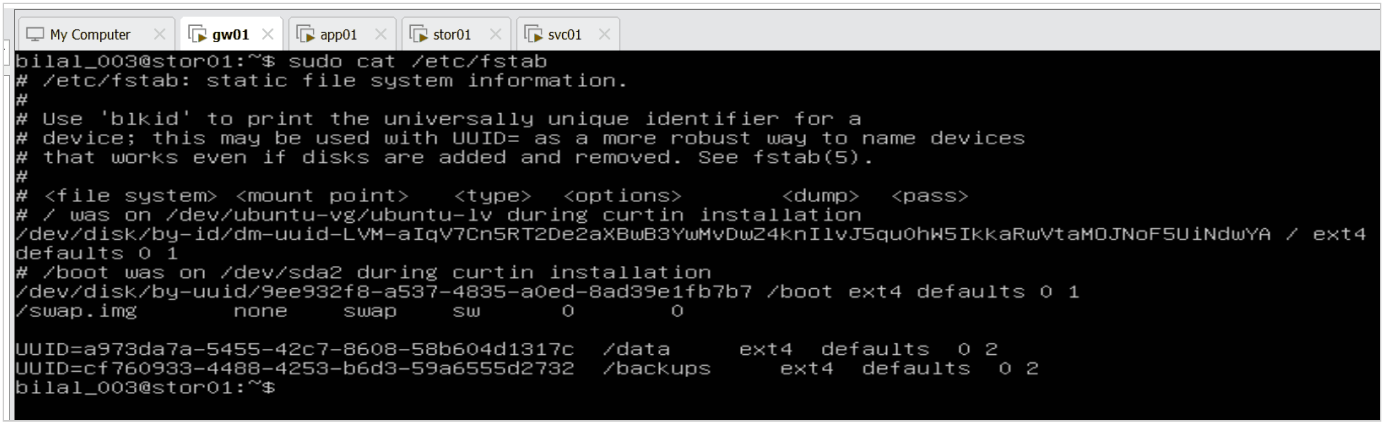
Seen	The `/data` line read `UUID=973da7a-…-42c7B8608-…` – the first a missing and a `-` typed as `B`.
Risk	At boot Linux would not find that volume and could stop in **emergency mode** – the server down because of one character.
Fix	Corrected in `nano /etc/fstab`, compared character by character with `blkid`.
Lesson	After every fstab change: `sudo mount -a` **before** a reboot. Better still, never type a UUID: `echo "UUID=$(sudo blkid -s UUID -o value /dev/vgdata/lvdata) /data ext4 defaults 0 2"`

📸 **Evidence – the wrong line:**



uuid typo

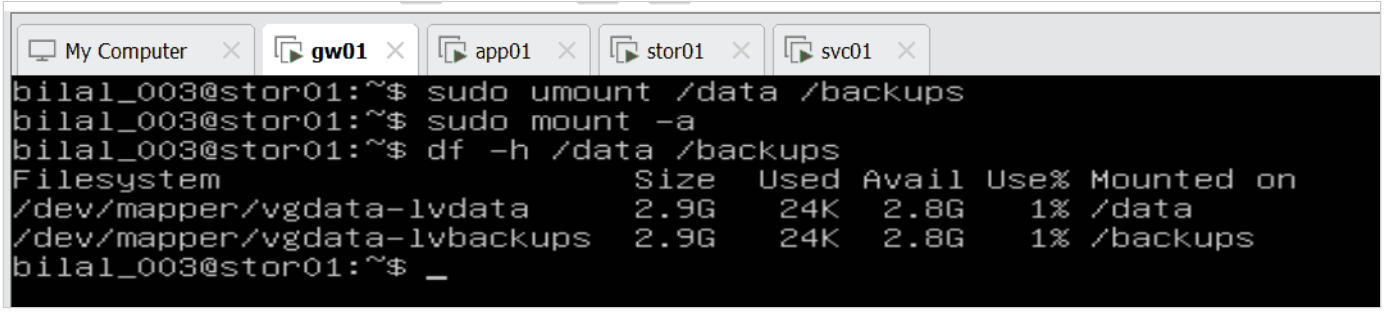
📸 **Evidence – fixed; both UUIDs match `blkid`:**



fstab fixed

```
sudo umount /data /backups
sudo mount -a              # no output = fstab is correct
df -h /data /backups
```

📸 **Evidence – mounted again from fstab alone:**



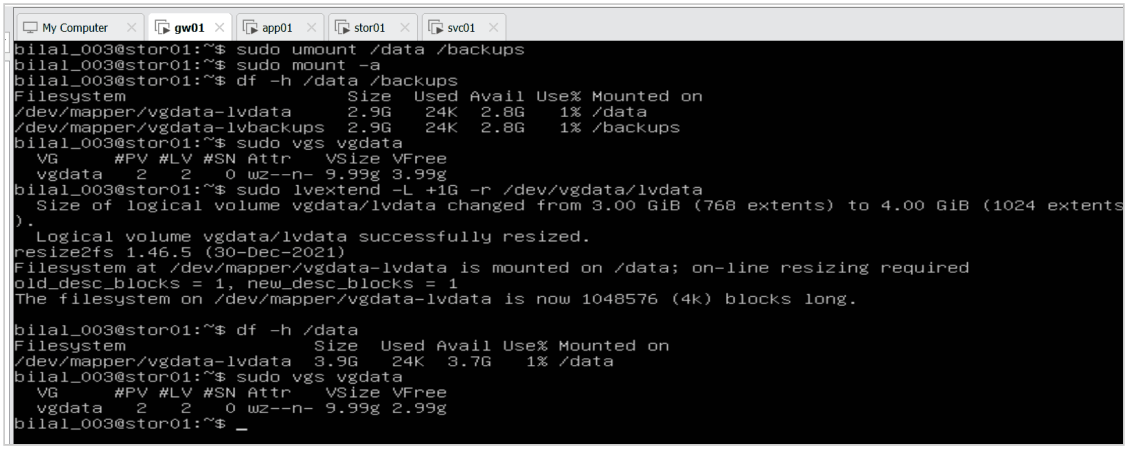
mount -a

5.5 Grow /data while it is running

💡 **The story:** a month later Acme's data has grown and `/data` is almost full. Without LVM: stop the app, add a disk, copy data. With LVM: take free space from the pool, live.

```
sudo vgs vgdata                                 # VFree 3.99 GB
sudo lvextend -L +1G -r /dev/vgdata/lvdata      # +1 GB; -r grows ext4 too
df -h /data                                     # 3.9 GB, no unmount, no downtime
sudo vgs vgdata                                 # VFree 2.99 GB
```

📸 **Evidence – 3.00 GiB (768 extents) → 4.00 GiB (1024 extents); "on-line resizing"; the pool's free space drops by 1 GB:**



lvextend

`-L +1G` (capital L) adds an exact size; on svc01 `-l +100%FREE` (small l) took all free space. LVM hands out space in 4 MB blocks called **extents**.

6. Phase 2 – NFS: app01 uses a folder on stor01

💡 **In plain words:** the app runs on app01, but its files should live on stor01, where the storage and backups are. **NFS** (Network File System) lets app01 use a stor01 folder over the network as if it were its own disk.

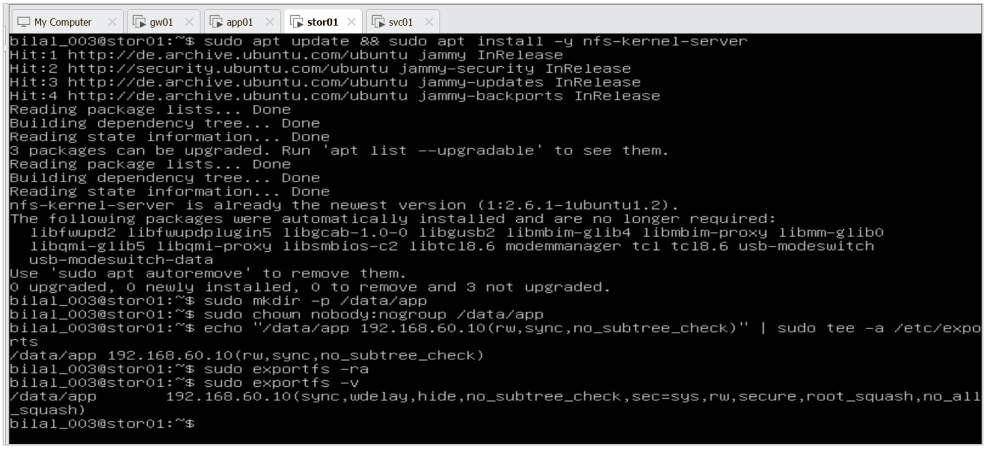
```
stor01 /data/app  ──NFS over the lab network──►  app01 /mnt/app
   (server: shares the folder)                  (client: mounts it)
```

6.1 stor01 – the server

```
sudo apt install -y nfs-kernel-server                          # already installed — nothing to download
sudo mkdir -p /data/app
sudo chown nobody:nogroup /data/app                            # neutral owner (see root_squash)
echo "/data/app 192.168.60.10(rw,sync,no_subtree_check)" | sudo tee -a /etc/exports
sudo exportfs -ra                                              # apply the share list
sudo exportfs -v                                               # what is shared, to whom
```

Part of the `/etc/exports` line	Meaning
`/data/app`	The folder to share
`192.168.60.10`	**Only app01** may mount it
`rw`	Read and write
`sync`	Write to disk before confirming – safer
`no_subtree_check`	Skip an old, slow check – standard choice
`root_squash` (default)	root on app01 is treated as `nobody` on stor01 – root on one machine is not root on the other

📸 **Evidence – shared with app01 only:**

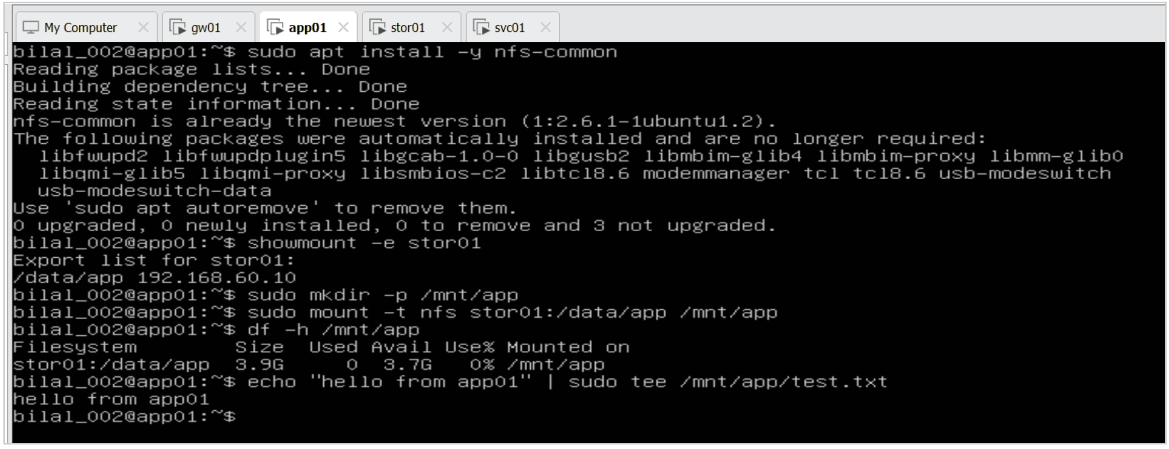


nfs server

6.2 app01 – the client, and the proof

```
sudo apt install -y nfs-common
showmount -e stor01                                  # what does stor01 share?
sudo mkdir -p /mnt/app
sudo mount -t nfs stor01:/data/app /mnt/app
df -h /mnt/app                                       # stor01:/data/app, 3.9 GB
echo "hello from app01" | sudo tee /mnt/app/test.txt   # write on app01
```

📸 **Evidence – app01:**



nfs client

```
cat /data/app/test.txt      # on stor01 — the same text
ls -l /data/app             # owner nobody:nogroup — root_squash at work
```

📸 **Evidence – stor01 sees the file at once:**



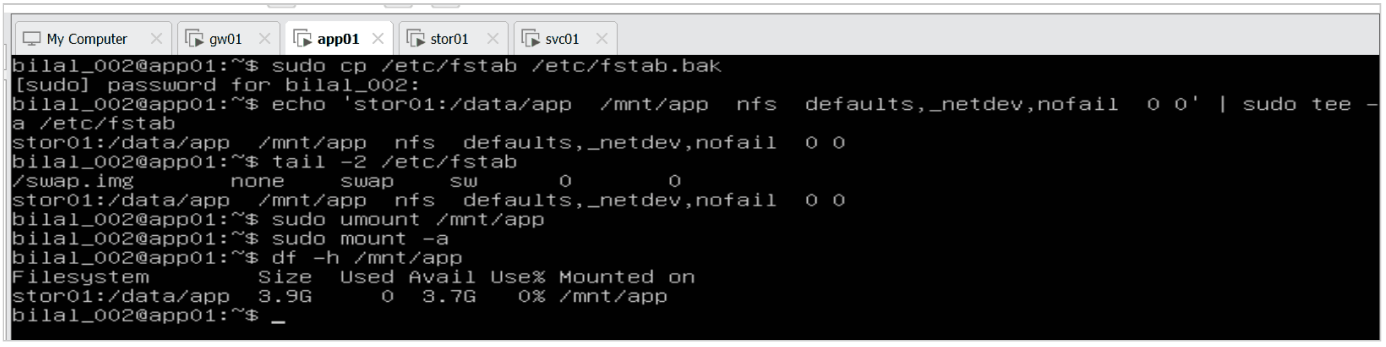
stor01 sees file

6.3 Permanent on app01

```
sudo cp /etc/fstab /etc/fstab.bak
echo 'stor01:/data/app  /mnt/app  nfs  defaults,_netdev,nofail  0 0' | sudo tee -a /etc/fstab
sudo umount /mnt/app
sudo mount -a
df -h /mnt/app
```

Option	Meaning
`_netdev`	A **network** disk – wait for the network before mounting
`nofail`	If stor01 is off, **app01 still boots**, just without `/mnt/app`
`0 0`	No dump, no disk check – stor01 checks its own disks

📸 **Evidence – mounted from fstab:**



app01 fstab

Lesson: a server must never fail to boot because **another** server is down.

Key understanding (in my own words)

Acme's application runs on app01, and its data and backups live on stor01. stor01 needed storage that can **grow** and an easy way to **share** files with app01.

LVM. `lsblk` showed two empty 5 GB disks, sdb and sdc. A raw disk cannot be used directly, so I built LVM layer by layer: `pvcreate` made both disks **physical volumes**; `vgcreate` put them into one **volume group**, `vgdata` – a 10 GB **pool** (a VG can be built from one or many PVs); `lvcreate` cut two 3 GB **logical volumes** from the pool, `lvdata` and `lvbackups`, so full backups cannot fill the app's space; `mkfs.ext4` formatted them and `mount` attached them to `/data` and `/backups`. Mounts disappear after a reboot, so I backed up `/etc/fstab` and added each volume's **UUID** from `blkid`. One wrong character in a UUID taught me to always test with `mount -a` before a reboot. Finally `lvextend -L +1G -r` grew `/data` from 3 to 4 GB while it was mounted – the main reason LVM exists.

NFS. On stor01 I installed the NFS server, created `/data/app` owned by `nobody:nogroup`, and shared it in `/etc/exports` with **only app01**, read/write. `root_squash` treats root from app01 as "nobody" on stor01. On app01 I installed the NFS client, checked the share with `showmount -e stor01` and mounted it on `/mnt/app`; a file written on app01 appeared at once on stor01. In app01's fstab I used `_netdev` (wait for the network) and `nofail` (boot even if stor01 is down).

In one line: stor01 now has storage that grows without downtime (LVM) and shares it with app01 over the network (NFS).

7. Phase 2 – backups: archive, mirror, versions, restore

💡 **In plain words:** a backup is a copy kept somewhere safe, so a lost file can come back. I made three kinds, all in `/backups` on stor01 (the `lvbackups` volume), and then proved they work by restoring a deleted file – through a real ticket.

```
stor01
/data/app/                      ← the original (shared with app01 over NFS)
/backups/
├── data-2026-10-18.tar.gz      ← ARCHIVE: one compressed file per day          (file)
├── mirror/                     ← MIRROR: exact copy of /data/app, only "now"   (real folder)
├── v1/  v2/  v3/  2026-10-19/  ← VERSIONS: one folder per run, history        (real folders)
└── latest -> 2026-10-19        ← shortcut to the newest version               (soft link)
```

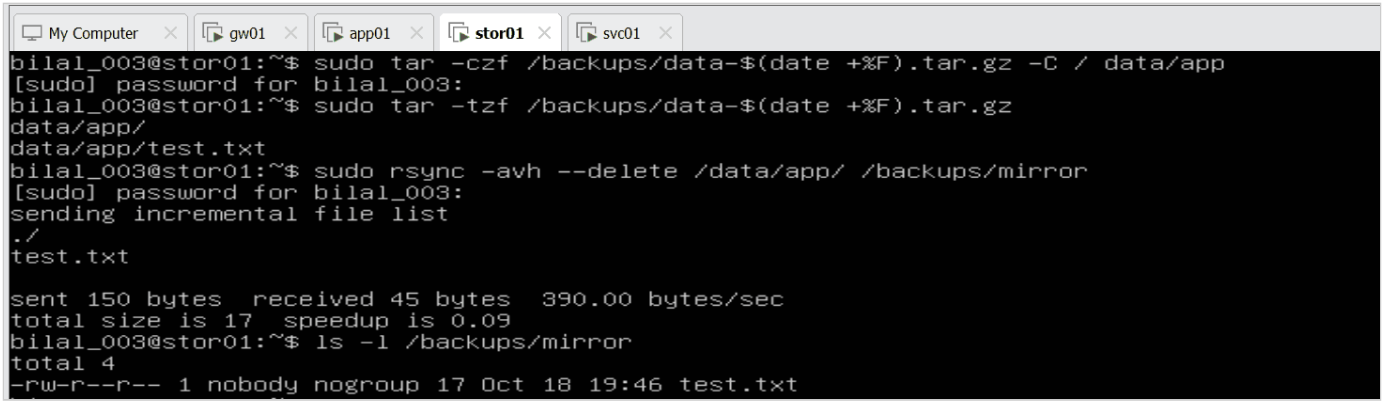
`/data`, `/backups` and `/data/app` are folders I created; `/mnt` is the standard empty folder for temporary mounts.

7.1 Archive (tar) and mirror (rsync)

```
sudo tar -czf /backups/data-$(date +%F).tar.gz -C / data/app   # pack /data/app into ONE dated, compressed file
tar -tzf /backups/data-$(date +%F).tar.gz                      # LIST what is inside – nothing is unpacked
sudo mkdir -p /backups/mirror                                   # a new, empty real folder
sudo rsync -avh --delete /data/app/ /backups/mirror/            # make it an exact copy of /data/app
```

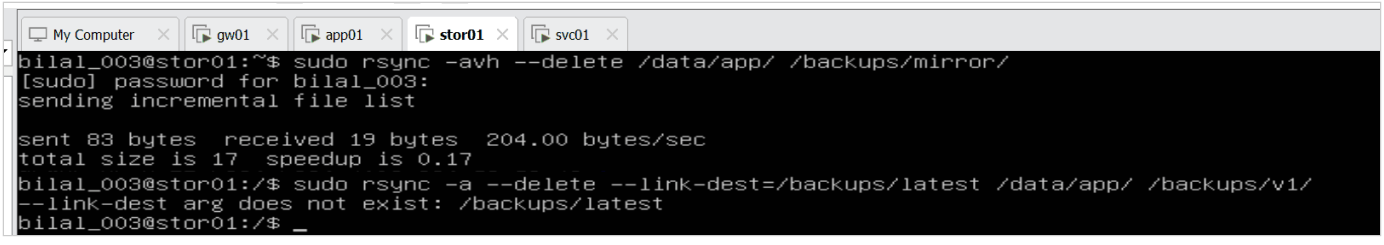
Part	Meaning
`tar -c -z -f`	**c**reate an archive, compress with g**z**ip, **f**ile name follows
`$(date +%F)`	today's date in the name, e.g. `2026-10-18`
`-C /`	go to `/` first, so the archive stores the path `data/app/…`; restore with `tar -xzf … -C /` puts files back in place
`rsync -a`	**a**rchive mode: subfolders, owners, permissions, dates kept
`-v` / `-h`	show each file / human-readable sizes
`--delete`	a file deleted in the source is removed from the **mirror** too – the source is never touched
`/data/app/`	trailing `/` = the *contents* of app

📸 **Evidence – the archive lists `data/app/test.txt`; rsync copies `test.txt`, keeping owner `nobody` and the original time:**



tar and mirror

📸 **Evidence – run again with nothing changed: rsync copies nothing (only "sent 83 bytes"). Then the first versioned run warns `--link-dest arg does not exist` – normal, there is no previous version yet:**



mirror rerun

	Archive (tar)	Mirror (rsync)
What it is	One compressed file per day	A normal folder with independent copies
History	Yes – one file per day	**No – only the state of the last run**
Restore one file	Unpack first	Copy it back

Hard link vs rsync mirror: a hard link is a second name for the **same** file (same inode) – a change appears instantly under both names, and damage hits both, so it is **not a backup**. The mirror holds **independent copies** (own inodes) and changes only when rsync runs – it **is** a backup, but `--delete` also copies mistakes at the next run.

7.2 Versions with hard links – history without wasting space

```
        sudo rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/v1/   # FROM /data/app  TO v1  COMPARE
WITH latest
        sudo ln -sfn /backups/v1 /backups/latest                                     # latest → v1
        echo "new file" | sudo tee /data/app/new.txt                                 # a change
        sudo rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/v2/
        sudo ln -sfn /backups/v2 /backups/latest                                     # latest → v2
        sudo rm /data/app/test.txt                                                   # the "accident"
        sudo rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/v3/
        sudo ln -sfn /backups/v3 /backups/latest                                     # latest → v3
        ls -l /backups
        ls -li /backups/v1 /backups/v2 /backups/v3
```

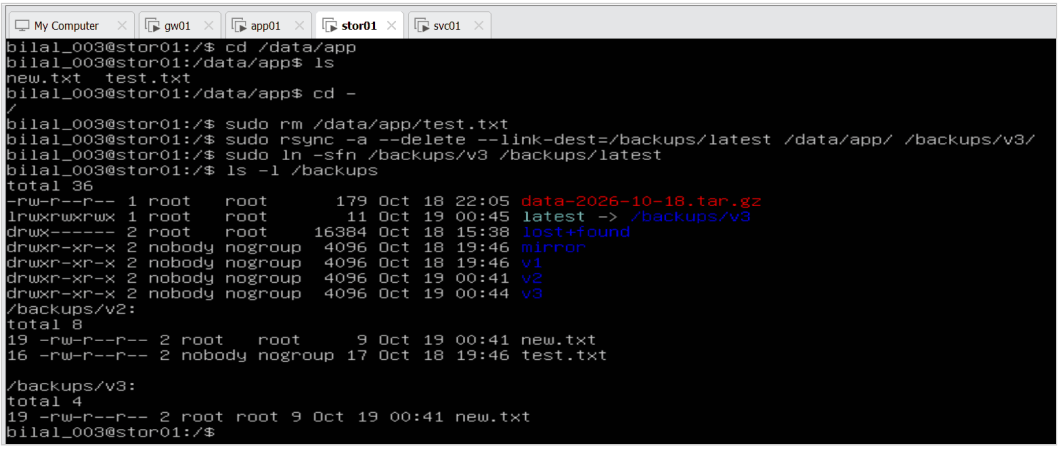
Piece	Job
`/data/app/`	**FROM** – the original
`/backups/v2/`	**TO** – the new version folder (rsync creates it)
`--link-dest=/backups/latest`	**COMPARE WITH** – the previous version: unchanged files become **hard links**, changed or new files are copied
`ln -sfn`	**s**oft link, **f**orce-replace, **n**o-follow – move the `latest` shortcut to the new version

📸 **Evidence – v1, then a new file, then v2:**

```
bilal_003@stor01:/$ sudo ln -sfn /backups/v1 /backups/latest
bilal_003@stor01:/$ echo "new file" | sudo tee /data/app/new.txt
new file
bilal_003@stor01:/$ sudo rsync -a --delete --link-dest=/backups/latest /data/app/ /backups/v2/
bilal_003@stor01:/$ sudo ln -sfn /backups/v2 /backups/latest
```

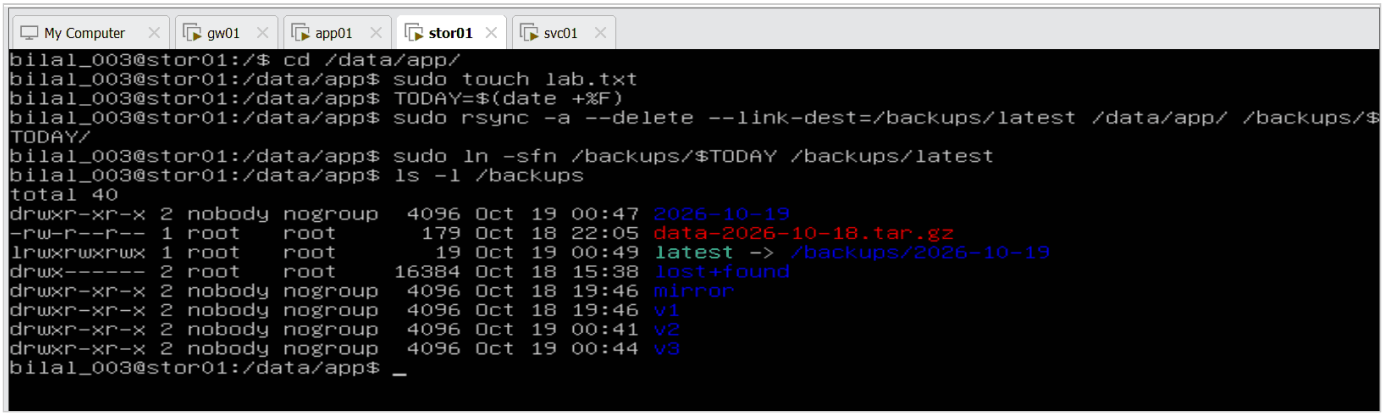
v1 v2

📸 **Evidence – after deleting test.txt, v3; `latest -> /backups/v3`; in v2 `test.txt` has inode 16 with link count 2 (shared with v1 – one copy on disk), `new.txt` inode 19 count 2 (shared with v3); v3 has no test.txt, but** v1 and v2 still do**:**



versions

📸 **Evidence – a real dated version, `/backups/2026-10-19`, and `latest` moved to it:**



dated backup

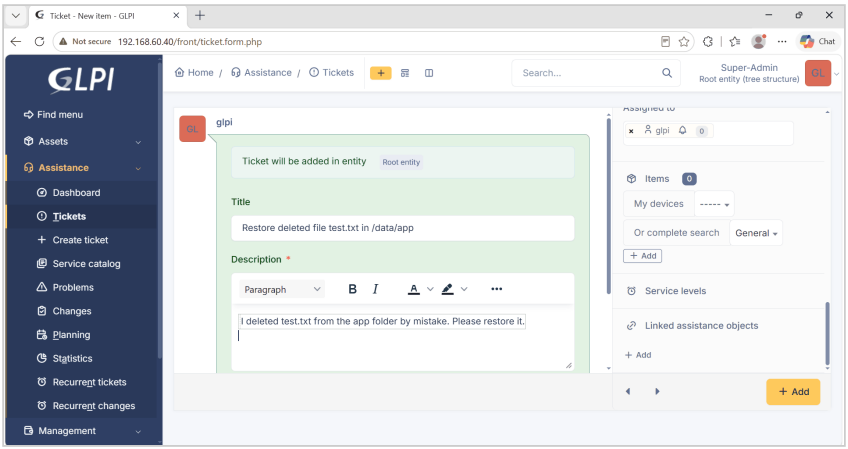
Run	latest before	What rsync does	latest after
1	does not exist (warning)	copies everything into v1	→ v1
2	→ v1	compares with v1: only changes copied, the rest hard-linked	→ v2
3	→ v2	compares with v2	→ v3

Old versions are **never changed** – that is the history. Hard links are safe here because they join backup to backup, not the original, and backup files are never edited.

7.3 Restore drill – through a ticket

💡 **The story:** Alice deletes `test.txt` by mistake and opens a ticket. Normal employees cannot touch `/backups` – it belongs to root and is handled by IT. The technician picks the version taken **before** the deletion and restores it.

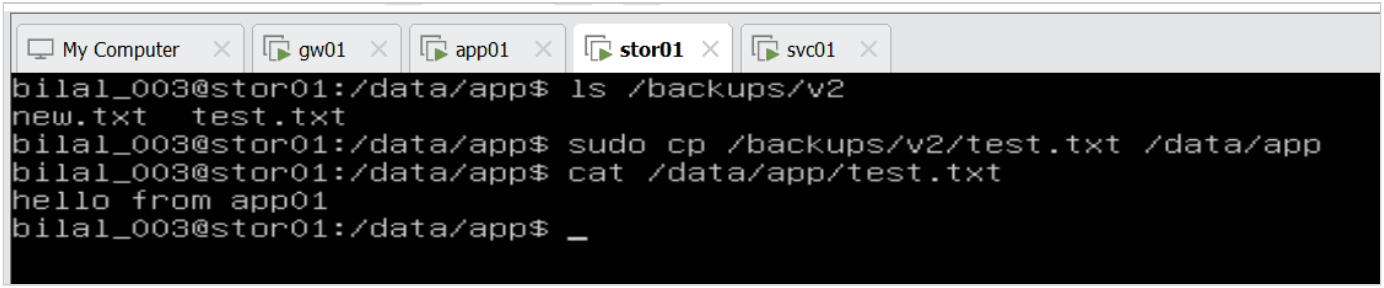
📸 **Evidence – Request ticket created (category *Storage & Backup*):**



ticket

```
ls /backups/v2                              # v3 no longer has the file – v2 does
sudo cp /backups/v2/test.txt /data/app      # restore: OUT of the backup, BACK to /data/app
cat /data/app/test.txt                      # "hello from app01"
```

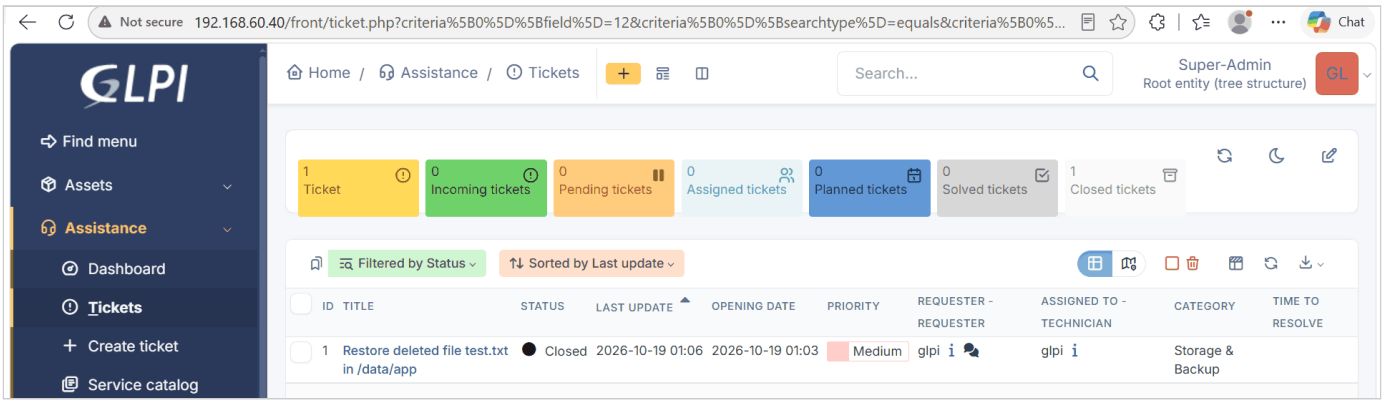
📸 **Evidence – restored and readable:**



restore

Solution written in the ticket: *Restored test.txt from backup version v2 (taken before the deletion) with `cp /backups/v2/test.txt /data/app/`. Content checked. Root cause: accidental deletion. No data lost.*

📸 **Evidence – ticket #1, Request, Storage & Backup,** Closed**:**



ticket closed

Key understanding (in my own words)

In `/backups` I keep three kinds of backup. The **archive** (`tar`) is one compressed **file** per day, for example `data-2026-10-18.tar.gz`; `tar -tzf` only lists what is inside. The **mirror**, `/backups/mirror`, is a **real folder**: `rsync -avh --delete` fills it with independent copies of `/data/app`. It is not live – it is updated only when rsync runs, and then copies only what is new or changed, like a patch; with `--delete`, files removed from the original are also removed from the mirror, so it only knows "now".

For **history**, each rsync run creates a **new version folder** (v1, v2, v3, or one per date). It takes the files **from** `/data/app`, writes them **to** the new folder, and **compares with** `/backups/latest`, the previous version. Changed or new files are really copied; unchanged files become **hard links** to the previous version, so every version looks complete but only the changes use space. Old versions are never changed. `/backups/latest` is the only **shortcut** (soft link); after each run `ln -sfn` moves it to the newest version, so the next run always compares with the most recent one.

A backup only counts once it has been restored. Alice deleted `test.txt` by mistake and opened a ticket. Normal employees cannot touch `/backups` – IT handles it at the administrative level. As the technician I found that v3, taken after the deletion, no longer had the file, but v2 did. I restored it with `sudo cp /backups/v2/test.txt /data/app/`, checked it with `cat`, wrote the solution in the ticket and closed it. The drill proves that the backups hold real files, that I can pick the right version, and that a lost file comes back working.

🗒️ All the files that make changes survive a reboot in this project: persistence cheat sheet.

8. Next steps

1. **Break test (Step 2.2):** from gw01, `sudo mount -t nfs stor01:/data/app /mnt` – must be refused: only app01 is in `/etc/exports`. Being reachable is not being allowed.
2. **Step 2.4 – LVM snapshot:** an undo button before risky changes – and why a snapshot is **not** a backup.
3. Remaining Phase 2 tickets (LVM build, grow /data, fstab incident, NFS share), then VMware snapshots `30-storage`.